

Rate Limiter

Команда №8:

Силевич Андрей
Бородатов Ярослав
Левина Яна
Тихомирова Дарья
Тастаков Александр

Введение

Rate Limiter — механизм ограничения частоты запросов к сервису.

Проблема:

отдельные клиенты или всплески трафика могут привести к перегрузке системы, увеличению задержек и снижению качества обслуживания пользователей.

Задача:

защитить сервис от перегрузки и обеспечить справедливый доступ к ресурсам.

Цель проекта:

Сравнить алгоритмы (*Token Bucket*, *Leaking Bucket* и *Sliding Window Log*) на разных паттернах нагрузки.

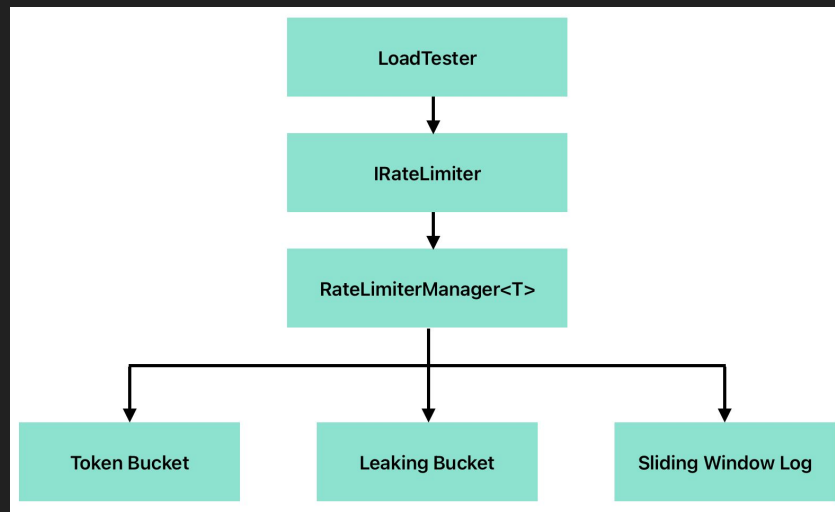
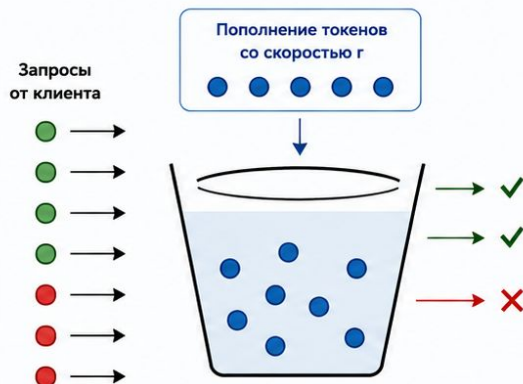


рис.1: Логическая архитектура проекта

Алгоритмы

Token Bucket

Токены накапливаются в ведре со скоростью g .
Каждый запрос потребляет 1 токен.
Если токенов нет — запрос отклоняется.

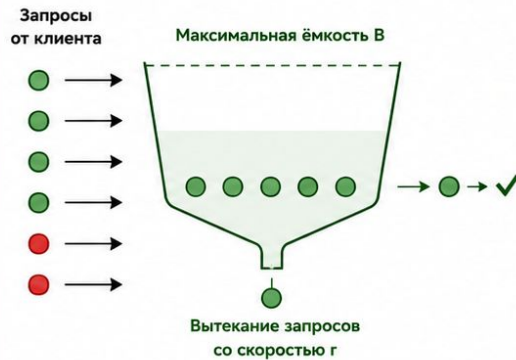


Характеристика:

допускает кратковременные всплески (burst), высокая производительность, малое потребление памяти

Leaking Bucket

Запросы помещаются в ведро (очередь).
Они "вытекают" с постоянной скоростью g .
При переполнении новые запросы отбрасываются.

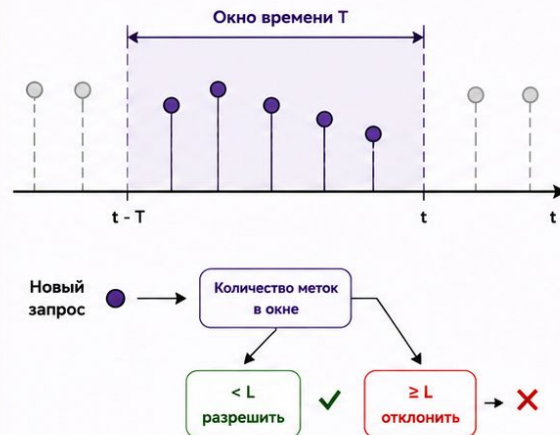


Характеристика:

сглаживает нагрузку, выдаёт запросы равномерно, возможны задержки

Sliding Window Log

Хранятся временные метки всех запросов за последний интервал времени T .
Новый запрос разрешается, если их меньше лимита L .



Временная метка запроса

Устаревшая метка (вне окна)

Характеристика:

наиболее точное ограничение, отсутствие эффекта границ окна, но требует больше памяти

Алгоритмы

Алгоритм	Token Bucket	Leaking Bucket	Sliding Window Log
Информация	Основная идея заключается в наличии виртуального ведра, содержащего некоторое кол-во токенов.	Моделирует ведро с отверстием, через которое запросы покидают очередь с постоянной скоростью.	Хранит временные метки всех запросов за заданный интервал.
Преимущества	- хорошо работает с burst-трафиком; - высокая производительность - малое потребление памяти.	- сглаживание нагрузки; - стабильная скорость обработки.	- максимальная точность; - отсутствие эффекта границ окна.
Недостатки	допускает кратковременные всплески нагрузки.	- плохо переносит burst-трафик; - возможны задержки запросов	- высокое потребление памяти; - сложность обслуживания большого кол-ва клиентов
Применение	- API Gateway; - балансировщики нагрузки; - системы аутентификации.	- сетевое оборудование; - маршрутизаторы; - телекоммуникационные системы.	- биллинговые системы; - критически важные API; - платёжные шлюзы.

Нагрузочные паттерны

Моделируют разные сценарии нагрузки: стабильный поток, всплески, дневной цикл и рост числа ключей

Uniform

Равномерный поток
запросов

Diurnal

Дневной цикл:
спад → пик → спад

Zombie

Много одноразовых
ключей, проверка TTL
и памяти

Burst

Резкие всплески
нагрузки

Sparse+Burst

Редкий трафик с
внезапными
всплесками

Исследование

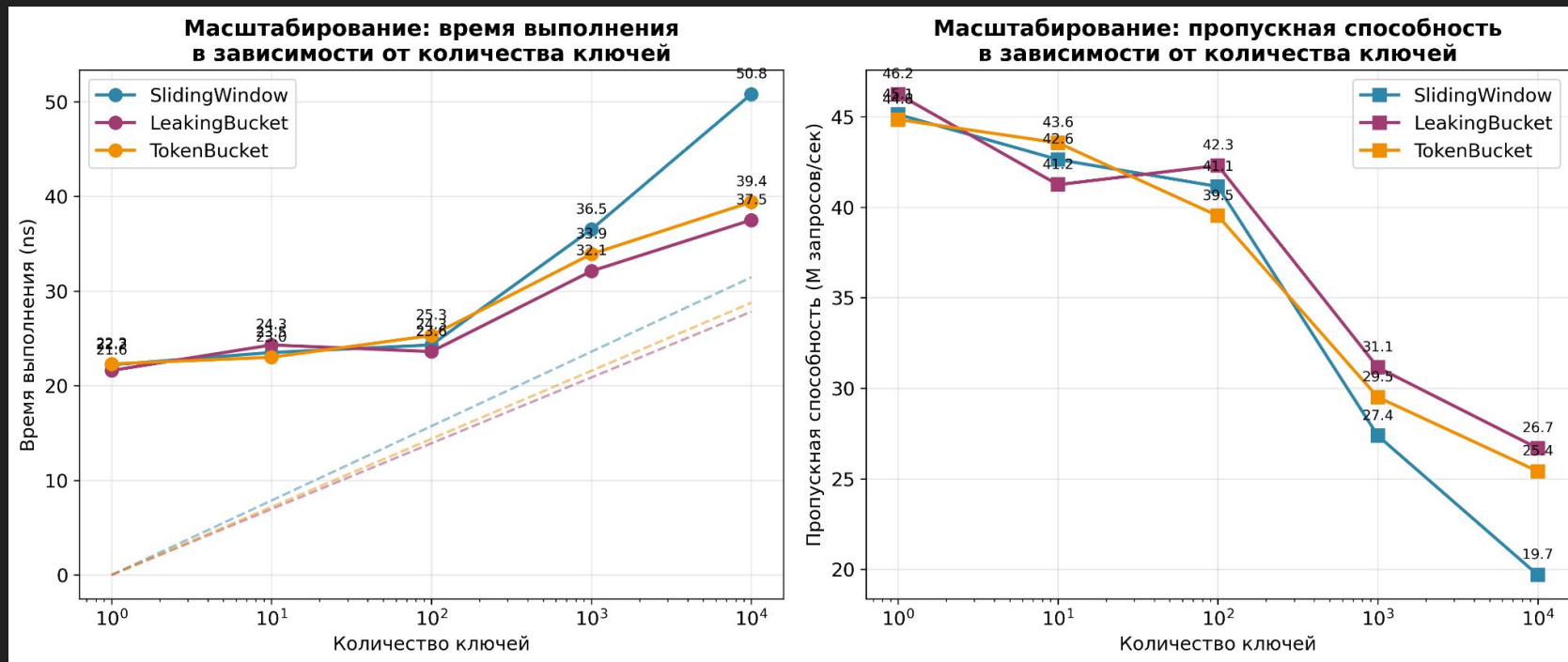


рис.2: графики зависимости от количества ключей

Исследование

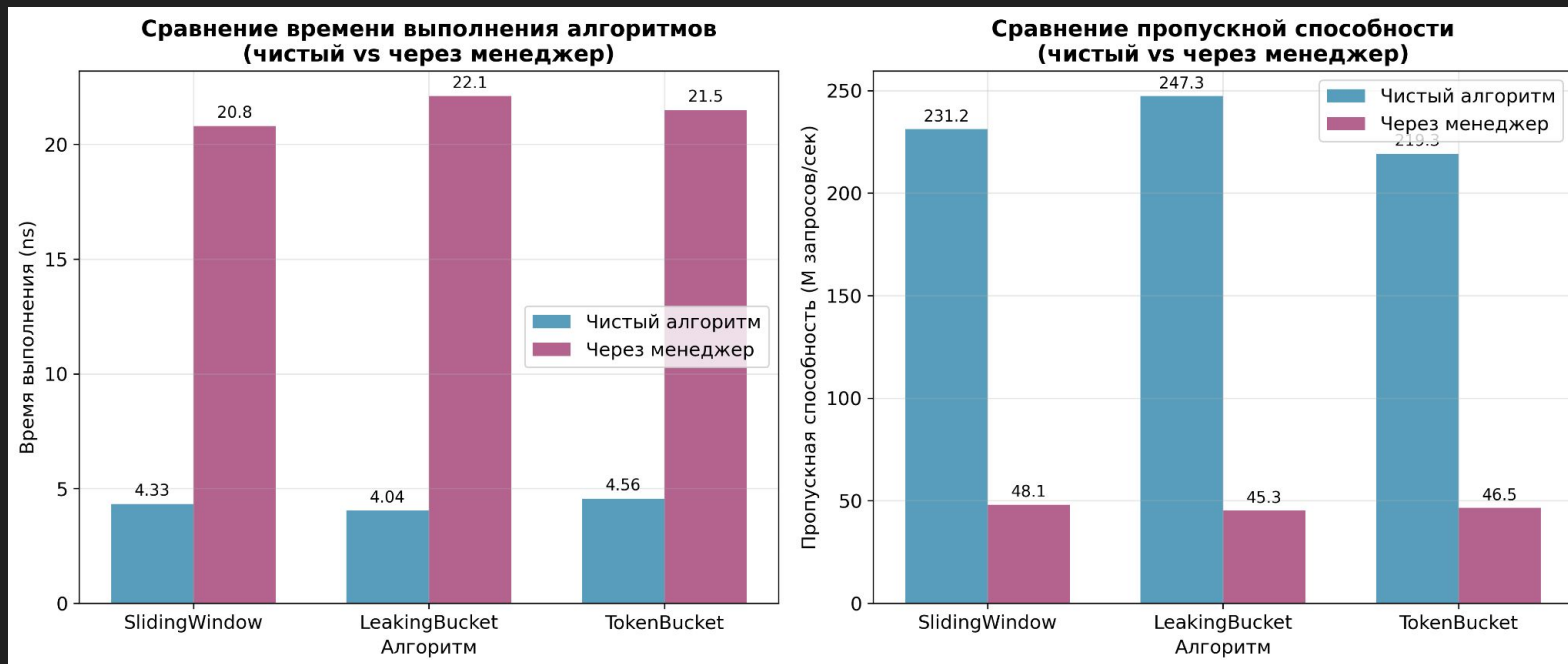


рис.3: сравнение алгоритмов

Исследование

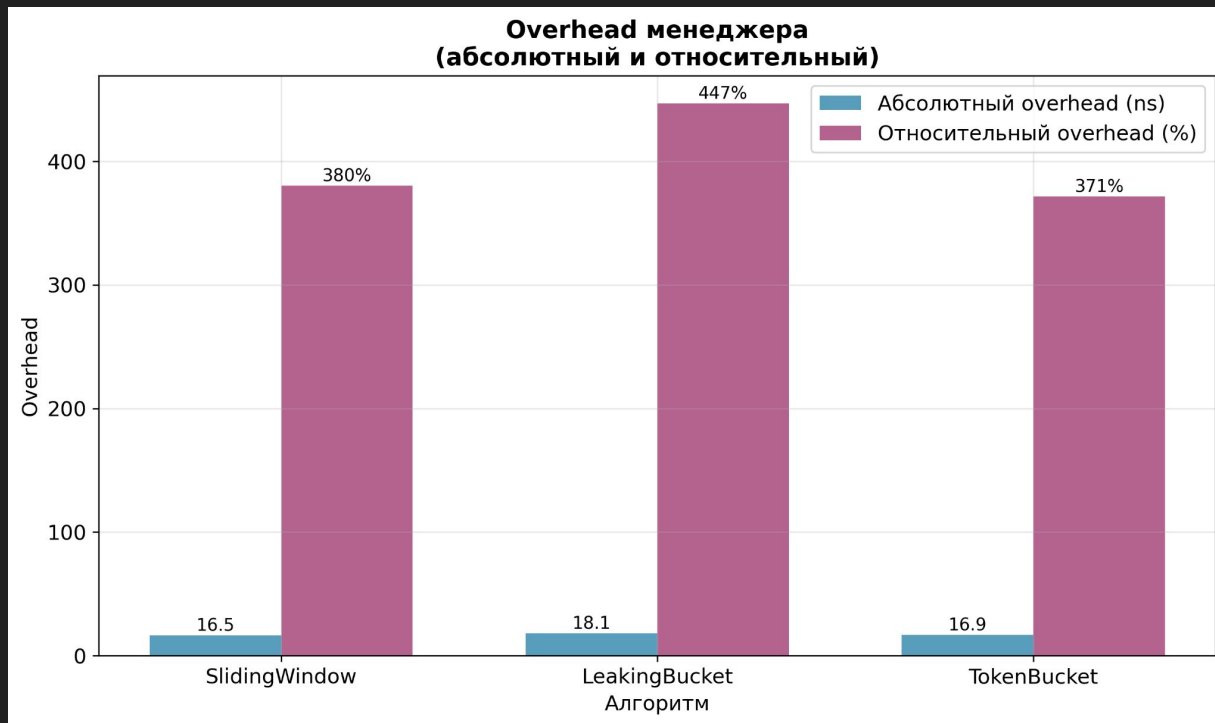


рис.4: overhead менеджера

Исследование

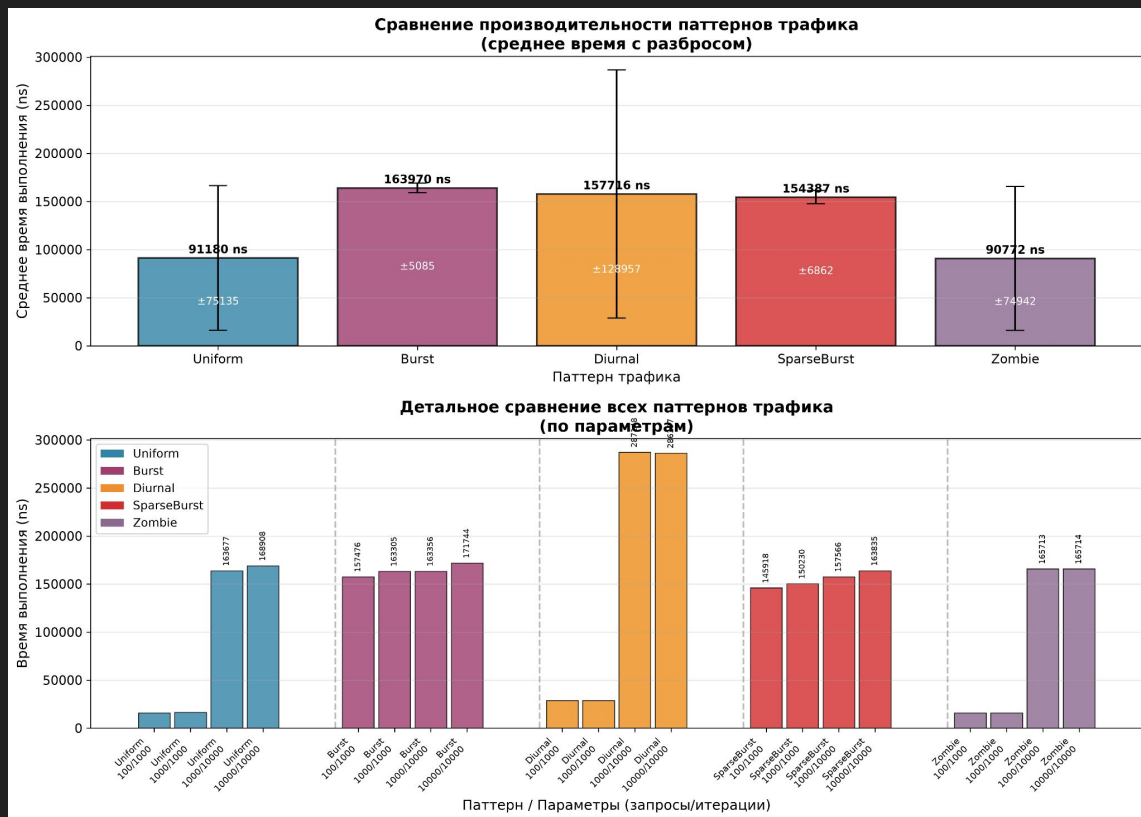


рис.5: сравнение паттернов

Спасибо за внимание